PLAN DE TRABAJO — SECUREBANK

Proyecto Final Integrador · QA · DevSecOps · Cloud · Automatización · Testing

Proyecto 1 — Plataforma FinTech de banca digital segura

Plan operativo dividido para 4 integrantes, incluyendo responsables, pasos de ejecución, dependencias, entregables y evidencias.

Base: documento original del curso y el plan de desarrollo elaborado por el equipo. El proyecto exige integración de frontend web, backend/API, PostgreSQL, Redis, RabbitMQ, aplicación móvil, Docker, Cloud/IaC, Ansible, Jenkins, pruebas y evidencia técnica.

1. Distribución de responsabilidades

Integrante	Responsabilidad principal	Área
Persona 1	Backend, API, PostgreSQL, lógica bancaria, Redis y RabbitMQ	Desarrollo Backend
Persona 2	Frontend web y aplicación móvil	Desarrollo Front/Mobile
Persona 3	Docker, Cloud, Terraform, Ansible y Jenkins	DevOps / Infraestructura
Persona 4	QA, Testing, Seguridad, Performance y evidencias	QA / DevSecOps

Regla transversal: ningún integrante trabaja aislado. Cada requisito debe seguir Implementación → Prueba → Evidencia → Documentación → Pull Request → Integración. Todos deben conocer la solución completa para la defensa.

2. Fase 0 — Organización inicial

Responsable: Persona 3. **Participan:** los cuatro.

Pasos:

1. Crear repositorio GitHub.
2. Crear ramas main y develop.
3. Proteger main y exigir Pull Request.
4. Crear Issues y etiquetas: backend, frontend, mobile, devops, qa, security, bug y documentation.
5. Crear README, estructura inicial y .env.example.
6. Crear carpetas para frontend, backend, mobile, infrastructure, ansible, tests, docs y reports.

Entregables: repositorio organizado, ramas, Issues, PR, README y estructura base.

3. Fase 1 — Diseño técnico

Persona 1: arquitectura backend, modelo de datos, endpoints, autenticación, autorización, transferencias, pagos, auditoría, errores e idempotencia.

Persona 2: diseño de pantallas web y móvil: login, dashboard, cuentas, movimientos, beneficiarios, transferencias, pagos, historial, saldo y logout.

Persona 3: seleccionar Azure + Terraform + Ansible o AWS + Pulumi + Ansible; diseñar infraestructura y ambiente reproducible.

Persona 4: iniciar plan maestro de pruebas, matriz de riesgos, criterios de entrada/salida y casos de prueba.

Cierre: arquitectura, modelo de datos, plan de pruebas inicial y matriz de riesgos.

4. Fase 2 — Backend + PostgreSQL

Responsable principal: Persona 1.

Paso 1: crear proyecto backend, migraciones y estructura de pruebas.

Paso 2: crear PostgreSQL con entidades mínimas: users, accounts, transactions, beneficiaries, transfers, payments y audit_logs.

Paso 3: implementar POST /api/auth/login con validación y JWT.

Paso 4: implementar GET /api/accounts.

Paso 5: implementar GET /api/accounts/{id}/transactions con autorización para evitar acceso a cuentas ajenas.

Paso 6: implementar GET/POST /api/beneficiaries.

Paso 7: implementar POST /api/transfers con saldo, autorización, transacción, idempotencia y auditoría.

Paso 8: implementar POST /api/payments.

Paso 9: registrar auditoría de operaciones.

Paso 10: crear pruebas unitarias de login, transferencias, saldo insuficiente, autorización, pagos e idempotencia.

Apoyo de Persona 4: preparar casos API positivos, negativos, autenticación, autorización y errores.

5. Fase 3 — Frontend web

Responsable: Persona 2.

1. Crear proyecto frontend.
2. Crear login y conectarlo con /api/auth/login.
3. Crear dashboard y consulta de cuentas.
4. Crear movimientos.
5. Crear beneficiarios.
6. Crear formulario de transferencia.
7. Crear pagos simulados.
8. Crear historial.
9. Integrar manejo de errores y estados de carga.
10. Ejecutar smoke test de los flujos principales.

Entregable: frontend integrado con la API y flujo principal funcionando.

6. Fase 4 — Aplicación móvil

Responsable: Persona 2.

1. Implementar login.
2. Implementar consulta de saldo.
3. Implementar transferencia.
4. Implementar historial.
5. Implementar logout.
6. Probar manualmente.
7. Preparar 5–8 flujos para Maestro, incluyendo al menos un flujo negativo.

7. Fase 5 — Redis

Responsable: Persona 1. Infraestructura: Persona 3. Evidencias: Persona 4.

1. Persona 3 agrega Redis al Docker Compose.
2. Persona 1 conecta el backend con Redis.

3. Implementar cache/sesiones/rate limiting según el diseño.
4. Demostrar HIT.
5. Demostrar MISS.
6. Demostrar TTL.
7. Demostrar invalidación.
8. Detener Redis y documentar el comportamiento de la aplicación.

Evidencias: hit.png, miss.png, ttl.png, invalidation.png y redis-down.png.

8. Fase 6 — RabbitMQ

Responsable: Persona 1. Infraestructura: Persona 3. QA: Persona 4.

1. Agregar RabbitMQ al ambiente.
2. Crear productor.
3. Crear eventos TransferCreated, TransferCompleted, TransferRejected, PaymentCreated y FraudAlert.
4. Crear consumidor.
5. Implementar retry/DLQ o estrategia equivalente.
6. Probar procesamiento correcto y manejo de errores.
7. Guardar evidencia del productor, consumidor y mensajes.

9. Fase 7 — Concurrencia e idempotencia

Responsable técnico: Persona 1. QA: Persona 4. Apoyo: Personas 2 y 3.

Escenario: preparar una cuenta con saldo conocido y generar transferencias simultáneas. Repetir la misma solicitud para simular doble envío.

1. Preparar cuenta origen y destinos.
2. Generar solicitudes concurrentes.
3. Repetir una misma solicitud.
4. Verificar saldo final.
5. Verificar que una operación idempotente no se procese dos veces.
6. Revisar auditoría y estados.
7. Observar RabbitMQ y Redis cuando corresponda.
8. Registrar tiempos, errores y resultados.

10. Fase 8 — Docker + ambiente QA

Responsable: Persona 3.

1. Crear Dockerfile de frontend.
2. Crear Dockerfile de backend.
3. Crear docker-compose.yml para frontend, backend, PostgreSQL, Redis y RabbitMQ.
4. Configurar variables mediante .env.example.
5. Levantar el ambiente con docker compose up -d.
6. Verificar conectividad entre servicios.
7. Documentar el procedimiento de reproducción.

Criterio: otra persona debe poder levantar el ambiente siguiendo la documentación.

11. Fase 9 — Cloud + Terraform + Ansible

Responsable: Persona 3.

1. Crear recursos en Azure mediante Terraform (si esa es la ruta elegida).
2. Crear red y recursos necesarios.
3. Ejecutar terraform init, plan y apply.
4. Demostrar una modificación controlada.
5. Ejecutar terraform destroy como evidencia de destrucción controlada cuando corresponda.
6. Crear inventario Ansible.
7. Crear playbooks para sistema operativo, Docker, reverse proxy y servicios requeridos.
8. Ejecutar Ansible y documentar resultados.

12. Fase 10 — Jenkins CI/CD

Responsable: Persona 3. Validación QA: Persona 4.

El Jenkinsfile debe ejecutar, como mínimo, en este orden:
1. Checkout → 2. Dependencias → 3. Lint/validaciones → 4. Unit tests → 5. SonarQube → 6. npm audit → 7. Dependency-Check → 8. Fortify → 9. Docker Build → 10. Deploy QA → 11. API tests → 12. Cypress → 13. Maestro → 14. OWASP ZAP → 15. Performance → 16. Quality Gate/criterios de salida → 17. Aprobación → 18. Deploy final.

Regla de bloqueo: una prueba crítica fallida, una vulnerabilidad crítica no aceptada o un Quality Gate rechazado debe impedir el despliegue final.

13. Fase 11 — QA y Seguridad

Responsable: Persona 4.

Insomnia: crear pruebas positivas, negativas, autenticación, autorización y errores. Probar especialmente acceso indebido a cuentas de otros usuarios.

Cypress: smoke, flujo crítico de transferencia, regresión y al menos un escenario negativo.

Maestro: 5–8 flujos relevantes de la aplicación móvil.

SonarQube: bugs, code smells, vulnerabilidades y Quality Gate.

Fortify: hallazgos SAST y severidad; documentar correcciones o aceptación justificada.

Dependency-Check + npm audit: detectar dependencias vulnerables y documentar tratamiento.

OWASP ZAP: ejecutar DAST contra el ambiente QA y clasificar hallazgos.

14. Fase 12 — Performance, usabilidad y accesibilidad

Responsable: Persona 4. Apoyo: Personas 1 y 3.

1. Definir umbrales antes de ejecutar.
2. Probar consultas de saldo y transferencias.
3. Ejecutar Load Test.
4. Ejecutar Stress Test.
5. Ejecutar Spike Test.
6. Registrar p95, tasa de error, throughput y recursos.
7. Realizar prueba de usabilidad con al menos 5 participantes y tareas definidas.
8. Ejecutar Lighthouse o axe para accesibilidad.

15. Fase 13 — Correcciones y regresión

Participan los cuatro.

1. Persona 4 registra cada defecto en GitHub Issues.
2. El responsable corrige el defecto en una rama.
3. Crear Pull Request.
4. Ejecutar pruebas.
5. Hacer merge después de la revisión.
6. Repetir pruebas de regresión.
7. Cerrar el defecto solamente con evidencia.

Cada bug debe contener: ID, título, descripción, pasos para reproducir, esperado, obtenido, severidad, prioridad, evidencia, responsable y estado.

16. Fase 14 — GO / NO-GO y defensa

Los cuatro consolidan métricas, evidencias y conclusión final. Preparan la demostración práctica y la presentación de 10–15 minutos.

Demostraciones que deben estar preparadas: ejecutar pipeline, provocar una prueba fallida, detener Redis o un consumidor RabbitMQ, mostrar un hallazgo de ZAP, explicar p95, recrear infraestructura y justificar GO/NO-GO.

GO: criterios de salida cumplidos y riesgos residuales documentados.
NO-GO: existe un bloqueo crítico no resuelto o un criterio de salida incumplido.

17. Entregables finales por integrante

Persona	Debe entregar
1 — Backend	API REST, PostgreSQL, autenticación, autorización, transferencias, pagos, auditoría, Redis, RabbitMQ, idempotencia, conc
2 — Front/Mobile	Frontend web completo, aplicación móvil, integración API, flujos funcionales y preparación de Cypress/Maestro.
3 — DevOps	GitHub, Docker, Compose, Azure/Terraform o AWS/Pulumi, Ansible, Jenkinsfile, pipeline y despliegue QA.
4 — QA/Security	Plan de pruebas, casos, Insomnia, Cypress, Maestro, SonarQube, Fortify, Dependency-Check, npm audit, ZAP, performanc

18. Cronograma de 14 semanas

Semana	Persona 1	Persona 2	Persona 3	Persona 4
1	Diseño API/BD	Diseño web/mobile	GitHub/IaC	Plan QA/riesgos
2	Auth + cuentas	Base frontend	Docker inicial	Casos API
3	Transferencias + pagos	Integración API	Ambiente QA	Unit/API testing
4	Correcciones API	Frontend completo	Docker Compose	Cypress
5	Ajustes API	Mobile	Ambiente mobile	Maestro
6	Redis/RabbitMQ	Integración	Servicios	Evidencias
7	Idempotencia	Validación UI	Ambiente pruebas	Concurrencia
8	Soporte	Soporte	Docker + QA	Integración QA
9	Soporte	Soporte	Azure + IaC + Ansible	Validación infraestructura
10	Soporte	Soporte	Jenkins	Quality Gates
11	Correcciones	Correcciones	Pipeline	Seguridad integral
12	Optimización API	Optimización UI	Recursos	Performance
13	Bugs	Bugs	Bugs infraestructura	Regresión
14	Defensa	Defensa	Defensa	Defensa/GO-NO-GO

19. Orden real de ejecución

GitHub → Arquitectura → Modelo BD → Backend básico → Frontend básico → Mobile básico → Docker → Redis → RabbitMQ → Transferencias → Idempotencia → Concurrencia → Ambiente QA → Cloud + IaC → Ansible → Jenkins → Insomnia →

Cypress → Maestro → SonarQube → Fortify → Dependency-Check → npm audit → ZAP → Performance → Usabilidad → Accesibilidad → Correcciones → Regresión → Métricas → Evidencias → GO/NO-GO → Defensa.

20. Estructura recomendada de evidencias

reports/
■■■ github/
■■■ backend/
■■■ frontend/
■■■ mobile/
■■■ redis/
■■■ rabbitmq/
■■■ docker/
■■■ terraform/
■■■ ansible/
■■■ jenkins/
■■■ sonar/
■■■ fortify/
■■■ dependency-check/
■■■ npm-audit/
■■■ zap/
■■■ insomnia/
■■■ cypress/
■■■ maestro/
■■■ performance/
■■■ usability/
■■■ accessibility/
■■■ defects/

Regla: no esperar al final para recopilar evidencias. Cada tarea debe generar su evidencia cuando se completa.

21. Lista de control antes de entregar

■ Repositorio GitHub completo y organizado.
■ Ramas, Pull Requests, Issues, Releases y protección de main.
■ Frontend, backend/API y aplicación móvil funcionales.
■ Dockerfiles y docker-compose.yml.
■ Jenkinsfile funcional.
■ Terraform para Azure o Pulumi para AWS.
■ Playbooks de Ansible.
■ Colección Insomnia con pruebas positivas, negativas y autorización.
■ Pruebas Cypress y flujos Maestro.
■ Pruebas de rendimiento.
■ Reportes SonarQube, Fortify, Dependency-Check, npm audit y ZAP.
■ Plan de pruebas formal.
■ Matriz de riesgos.
■ Casos de prueba y evidencias.
■ Registro de defectos.
■ Informe final con métricas y decisión GO/NO-GO.
■ Presentación profesional y demostración en vivo.

Nota: Este documento organiza los requisitos del Proyecto 1 — SecureBank en una distribución operativa para cuatro integrantes. No sustituye el documento original del curso; sirve como guía de ejecución y coordinación del equipo.